

This project analyzes a real-world retail sales dataset containing more than 50,000 sales records.

The objective is to clean the data, explore sales and profit patterns, identify important business trends, and present the findings using meaningful visualizations.

Domain Retail / E-commerce

Tools Used

- Python
- Pandas
- NumPy
- Matplotlib
- Seaborn

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
print("Libraries imported successfully!")
```

```
Libraries imported successfully!
```

```
df = pd.read_csv("task4.csv")
print("Dataset loaded successfully!")
```

```
Dataset loaded successfully!
```

```
df.head()
```

	Order_ID	Order_Date	Customer_ID	Region	City	Customer_Segment	Category	Product	Sales_Channel	Payment_Method
0	ORD1000000	2023-02-09	CUST10000	Central	Bhopal	Small Business	Electronics	Monitor	Website	Debit Card
1	ORD1000001	2024-04-18	CUST10001	South	Coimbatore	Small Business	Home & Kitchen	Coffee Maker	Website	Wallet
2	ORD1000002	2023-08-21	CUST10002	South	Coimbatore	Consumer	Sports	Cricket Bat	Mobile App	Debit Card
3	ORD1000003	2023-07-18	CUST10003	North	Jaipur	Corporate	Electronics	Monitor	Website	UPI
4	ORD1000004	2023-02-13	CUST10004	Central	Bhopal	Consumer	Home & Kitchen	Storage Box	Website	Credit Card

5 rows × 26 columns

```
df.shape
```

```
(28564, 26)
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 28564 entries, 0 to 28563
Data columns (total 26 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Order_ID              28564 non-null  object
1   Order_Date            28564 non-null  object
2   Customer_ID           28564 non-null  object
3   Region                28378 non-null  object
4   City                  28563 non-null  object
5   Customer_Segment      28563 non-null  object
6   Category              28563 non-null  object
7   Product               28563 non-null  object
8   Sales_Channel         28563 non-null  object
9   Payment_Method        28338 non-null  object
10  Shipping_Mode         28563 non-null  object
11  Quantity              28563 non-null  float64
12  Unit_Price            28563 non-null  float64
```

```

13 Discount_Pct      28457 non-null float64
14 Gross_Sales       28563 non-null float64
15 Discount_Amount   28563 non-null float64
16 Net_Sales         28563 non-null float64
17 Profit            28563 non-null float64
18 Profit_Margin_Pct 28563 non-null float64
19 Delivery_Days     28563 non-null float64
20 Customer_Rating   28220 non-null float64
21 Order_Status      28563 non-null object
22 Return_Flag       28563 non-null float64
23 Month             28563 non-null float64
24 Quarter           28563 non-null float64
25 Year              28563 non-null float64

```

```
dtypes: float64(14), object(12)
```

```
memory usage: 5.7+ MB
```

```
print(df.columns)
```

```

Index(['Order_ID', 'Order_Date', 'Customer_ID', 'Region', 'City',
      'Customer_Segment', 'Category', 'Product', 'Sales_Channel',
      'Payment_Method', 'Shipping_Mode', 'Quantity', 'Unit_Price',
      'Discount_Pct', 'Gross_Sales', 'Discount_Amount', 'Net_Sales', 'Profit',
      'Profit_Margin_Pct', 'Delivery_Days', 'Customer_Rating', 'Order_Status',
      'Return_Flag', 'Month', 'Quarter', 'Year'],
      dtype='object')

```

```
df.describe()
```

	Quantity	Unit_Price	Discount_Pct	Gross_Sales	Discount_Amount	Net_Sales	Profit	Profit_Margin_Pct	De
count	28563.000000	28563.000000	28457.000000	28563.000000	28563.000000	28563.000000	28563.000000	28563.000000	
mean	2.160067	9275.627508	12.061356	19967.521020	2400.222406	17567.298626	3347.808950	23.665181	
std	1.382290	13497.074455	8.731945	36653.667885	5728.640835	32387.412756	5410.214853	7.026851	
min	1.000000	528.000000	0.000000	528.000000	0.000000	343.200000	111.860000	12.800000	
25%	1.000000	1981.740000	5.000000	3590.830000	215.095000	3160.070000	808.270000	17.790000	
50%	2.000000	4395.330000	10.000000	7821.270000	728.220000	6795.020000	1612.490000	22.300000	
75%	3.000000	8866.235000	20.000000	19478.555000	2139.945000	17190.605000	3475.190000	29.260000	
max	7.000000	72772.610000	35.000000	509196.590000	129178.780000	458276.930000	74880.790000	40.800000	

```
df.describe(include="object")
```

	Order_ID	Order_Date	Customer_ID	Region	City	Customer_Segment	Category	Product	Sales_Channel	Payment_Me
count	28564	28564	28564	28378	28563	28563	28563	28563	28563	28
unique	28564	1096	8501	5	20	4	6	36	4	
top	ORD1028563	2025-02-21	CUST13062	South	Coimbatore	Consumer	Electronics	Laptop	Website	
freq	1	43	4	7719	1956	15754	6557	1118	10374	9

```
df.nunique()
```

	0
Order_ID	28564
Order_Date	1096
Customer_ID	8501
Region	5
City	20
Customer_Segment	4
Category	6
Product	36
Sales_Channel	4
Payment_Method	6
Shipping_Mode	3
Quantity	7
Unit_Price	28092
Discount_Pct	8
Gross_Sales	28310
Discount_Amount	23252
Net_Sales	28356
Profit	27676
Profit_Margin_Pct	2770
Delivery_Days	9
Customer_Rating	35
Order_Status	4
Return_Flag	2
Month	12
Quarter	4
Year	3

dtype: int64

```
df.isnull().sum()
```

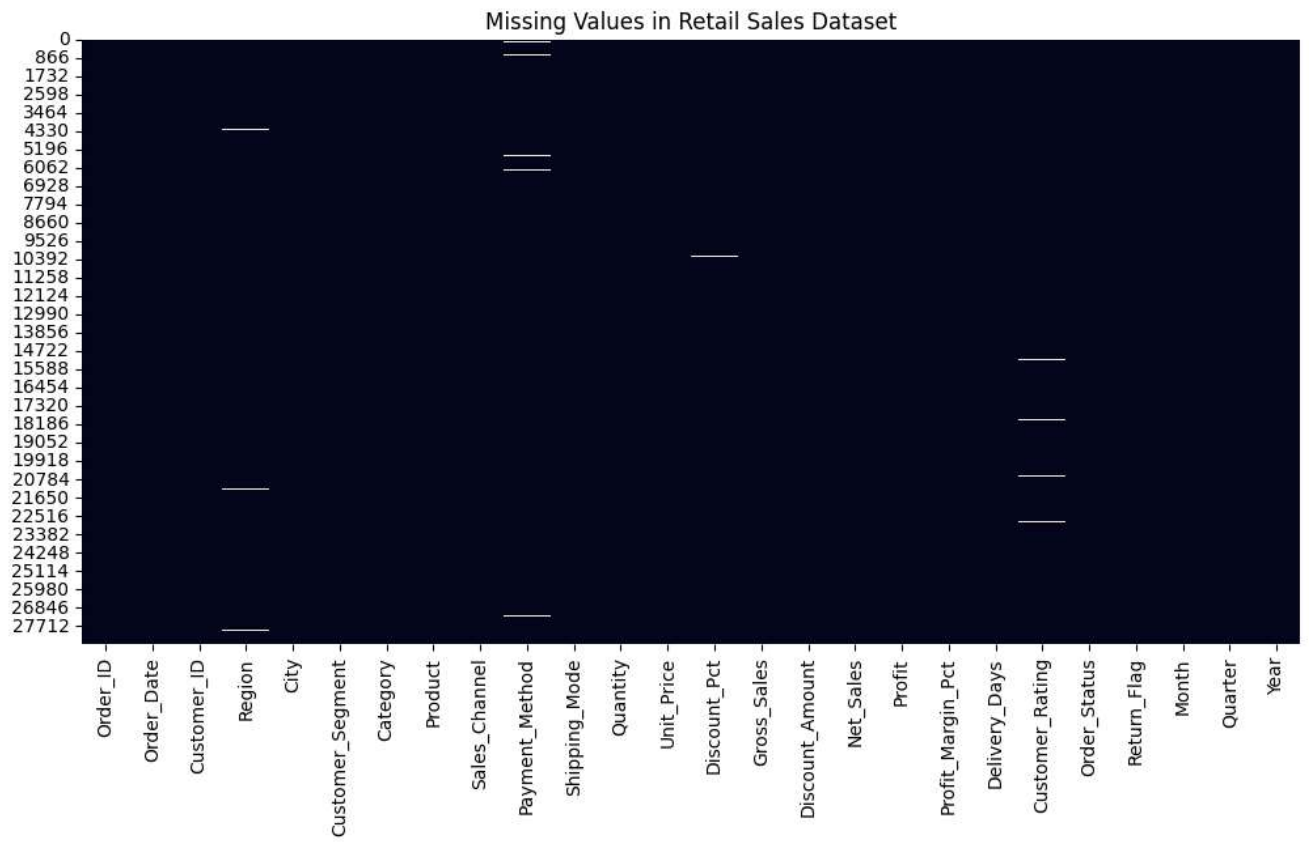
	0
Order_ID	0
Order_Date	0
Customer_ID	0
Region	186
City	1
Customer_Segment	1
Category	1
Product	1
Sales_Channel	1
Payment_Method	226
Shipping_Mode	1
Quantity	1
Unit_Price	1
Discount_Pct	107
Gross_Sales	1
Discount_Amount	1
Net_Sales	1
Profit	1
Profit_Margin_Pct	1
Delivery_Days	1
Customer_Rating	344
Order_Status	1
Return_Flag	1
Month	1
Quarter	1
Year	1

dtype: int64

```
plt.figure(figsize=(12,6))

sns.heatmap(df.isnull(), cbar=False)

plt.title("Missing Values in Retail Sales Dataset")
plt.show()
```



```
df["Region"] = df["Region"].fillna(df["Region"].mode()[0])
```

```
df["Payment_Method"] = df["Payment_Method"].fillna(
    df["Payment_Method"].mode()[0]
)
```

```
df["Discount_Pct"] = df["Discount_Pct"].fillna(
    df["Discount_Pct"].median()
)
```

```
df["Customer_Rating"] = df["Customer_Rating"].fillna(
    df["Customer_Rating"].median()
)
```

```
df.isnull().sum()
```

	0
Order_ID	0
Order_Date	0
Customer_ID	0
Region	0
City	1
Customer_Segment	1
Category	1
Product	1
Sales_Channel	1
Payment_Method	0
Shipping_Mode	1
Quantity	1
Unit_Price	1
Discount_Pct	0
Gross_Sales	1
Discount_Amount	1
Net_Sales	1
Profit	1
Profit_Margin_Pct	1
Delivery_Days	1
Customer_Rating	0
Order_Status	1
Return_Flag	1
Month	1
Quarter	1
Year	1

dtype: int64

```
df.duplicated().sum()
```

```
np.int64(0)
```

```
df = df.drop_duplicates()
```

```
print("Duplicates remaining:", df.duplicated().sum())
```

```
Duplicates remaining: 0
```

```
df["Order_Date"] = pd.to_datetime(df["Order_Date"])
```

```
df["Order_Date"].dtype
```

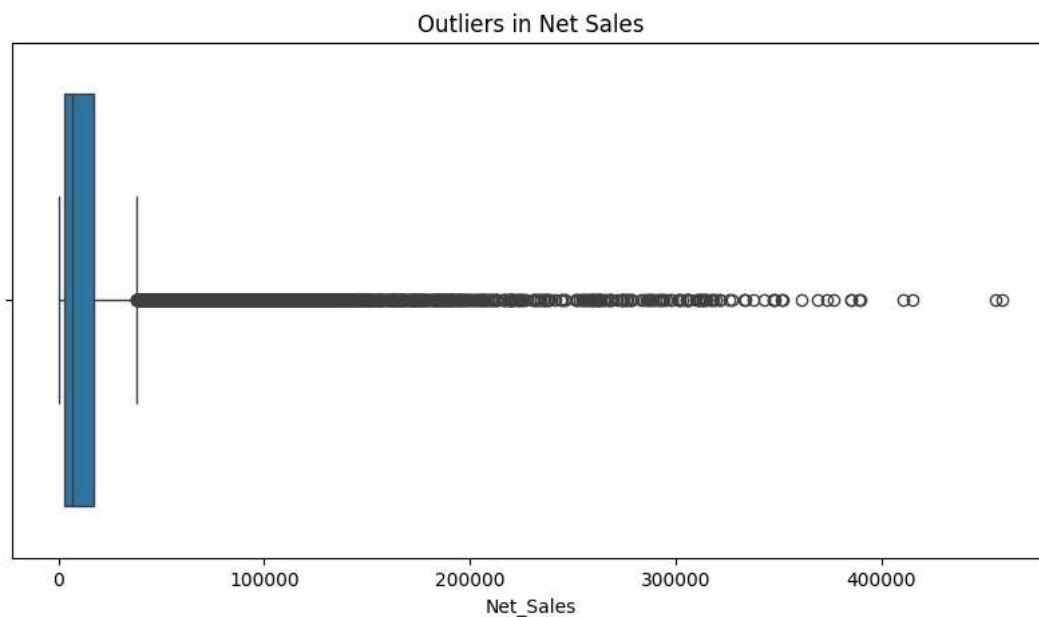
```
dtype('<M8[ns]')
```

```
numeric_columns = [
    "Quantity",
    "Unit_Price",
    "Gross_Sales",
    "Discount_Amount",
    "Net_Sales",
    "Profit",
    "Profit_Margin_Pct",
    "Delivery_Days",
    "Customer_Rating"
```

```
]
df[numeric_columns].describe()
```

	Quantity	Unit_Price	Gross_Sales	Discount_Amount	Net_Sales	Profit	Profit_Margin_Pct	Delivery_Days
count	28563.000000	28563.000000	28563.000000	28563.000000	28563.000000	28563.000000	28563.000000	28563.000000
mean	2.160067	9275.627508	19967.521020	2400.222406	17567.298626	3347.808950	23.665181	3.796485
std	1.382290	13497.074455	36653.667885	5728.640835	32387.412756	5410.214853	7.026851	1.983529
min	1.000000	528.000000	528.000000	0.000000	343.200000	111.860000	12.800000	0.000000
25%	1.000000	1981.740000	3590.830000	215.095000	3160.070000	808.270000	17.790000	2.000000
50%	2.000000	4395.330000	7821.270000	728.220000	6795.020000	1612.490000	22.300000	4.000000
75%	3.000000	8866.235000	19478.555000	2139.945000	17190.605000	3475.190000	29.260000	5.000000
max	7.000000	72772.610000	509196.590000	129178.780000	458276.930000	74880.790000	40.800000	8.000000

```
plt.figure(figsize=(10,5))
sns.boxplot(x=df["Net_Sales"])
plt.title("Outliers in Net Sales")
plt.show()
```



```
df["Region"] = df["Region"].fillna(df["Region"].mode()[0])
df["Payment_Method"] = df["Payment_Method"].fillna(
    df["Payment_Method"].mode()[0]
)
```

```
df["Discount_Pct"] = df["Discount_Pct"].fillna(
    df["Discount_Pct"].median()
)
df["Customer_Rating"] = df["Customer_Rating"].fillna(
    df["Customer_Rating"].median()
)
```

```
df.isnull().sum()
```

	0
Order_ID	0
Order_Date	0
Customer_ID	0
Region	0
City	1
Customer_Segment	1
Category	1
Product	1
Sales_Channel	1
Payment_Method	0
Shipping_Mode	1
Quantity	1
Unit_Price	1
Discount_Pct	0
Gross_Sales	1
Discount_Amount	1
Net_Sales	1
Profit	1
Profit_Margin_Pct	1
Delivery_Days	1
Customer_Rating	0
Order_Status	1
Return_Flag	1
Month	1
Quarter	1
Year	1

dtype: int64

```
df.duplicated().sum()
```

```
np.int64(0)
```

```
df = df.drop_duplicates()
```

```
df["Day"] = df["Order_Date"].dt.day
df["Weekday"] = df["Order_Date"].dt.day_name()
```

```
numeric_columns = [
    "Quantity",
    "Unit_Price",
    "Discount_Pct",
    "Gross_Sales",
    "Discount_Amount",
    "Net_Sales",
    "Profit",
    "Profit_Margin_Pct",
    "Delivery_Days",
    "Customer_Rating"
]
```

```
Q1 = df["Net_Sales"].quantile(0.25)
Q3 = df["Net_Sales"].quantile(0.75)
```

```
IQR = Q3 - Q1
```

```
lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR

outliers = df[
    (df["Net_Sales"] < lower) |
    (df["Net_Sales"] > upper)
]

print("Number of outliers:", len(outliers))
```

Number of outliers: 3273

```
df.to_csv("cleaned_retail_sales.csv", index=False)
```

```
total_orders = df["Order_ID"].nunique()
print("Total Orders:", total_orders)
```

Total Orders: 28564

```
total_sales = df["Net_Sales"].sum()
print("Total Net Sales:", total_sales)
```

Total Net Sales: 501774750.65000004

```
total_profit = df["Profit"].sum()
print("Total Profit:", total_profit)
```

Total Profit: 95623467.03

```
average_order_value = df["Net_Sales"].mean()
print("Average Order Value:", average_order_value)
```

Average Order Value: 17567.298625844625

```
average_rating = df["Customer_Rating"].mean()
print("Average Customer Rating:", average_rating)
```

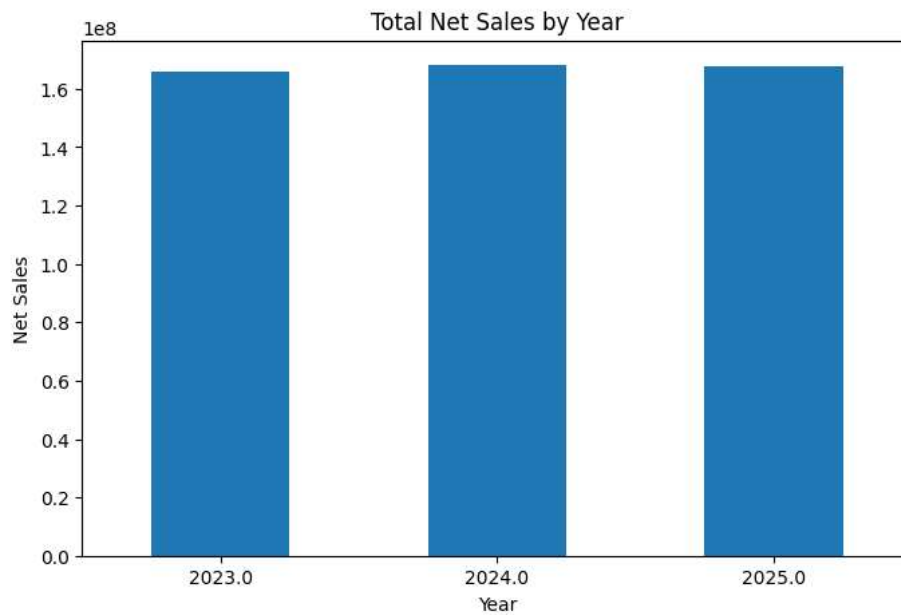
Average Customer Rating: 4.020497829435653

```
return_rate = df["Return_Flag"].mean() * 100
print("Return Rate:", return_rate)
```

Return Rate: 5.188530616531877

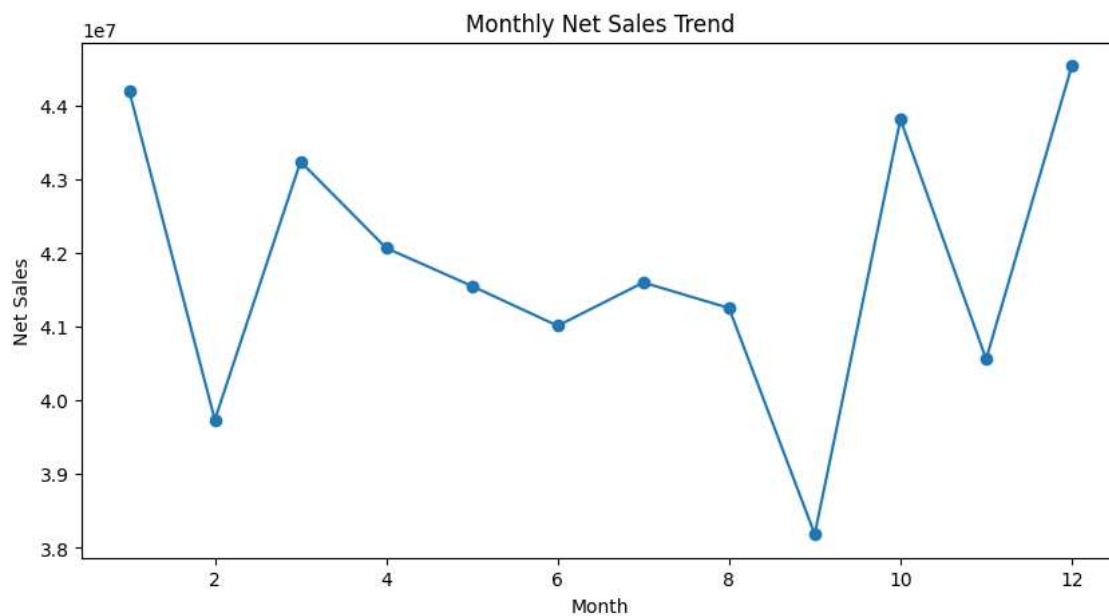
```
yearly_sales = df.groupby("Year")["Net_Sales"].sum()

plt.figure(figsize=(8,5))
yearly_sales.plot(kind="bar")
plt.title("Total Net Sales by Year")
plt.xlabel("Year")
plt.ylabel("Net Sales")
plt.xticks(rotation=0)
plt.show()
```



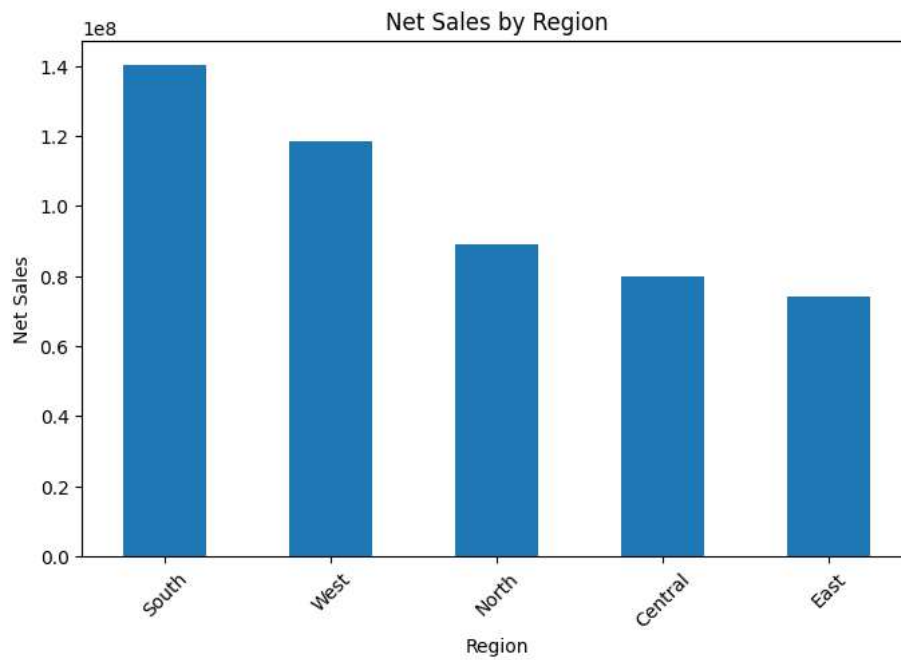
```
monthly_sales = df.groupby("Month")["Net_Sales"].sum()
```

```
plt.figure(figsize=(10,5))
monthly_sales.plot(kind="line", marker="o")
plt.title("Monthly Net Sales Trend")
plt.xlabel("Month")
plt.ylabel("Net Sales")
plt.show()
```



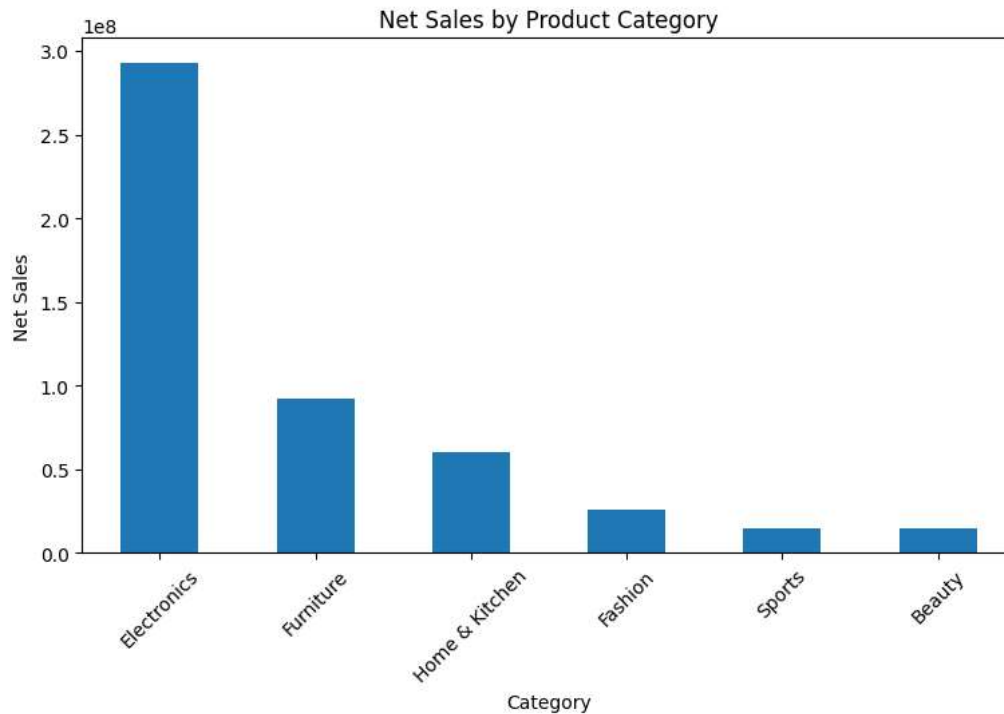
```
region_sales = df.groupby("Region")["Net_Sales"].sum().sort_values(ascending=False)
```

```
plt.figure(figsize=(8,5))
region_sales.plot(kind="bar")
plt.title("Net Sales by Region")
plt.xlabel("Region")
plt.ylabel("Net Sales")
plt.xticks(rotation=45)
plt.show()
```



```
category_sales = df.groupby("Category")["Net_Sales"].sum().sort_values(ascending=False)

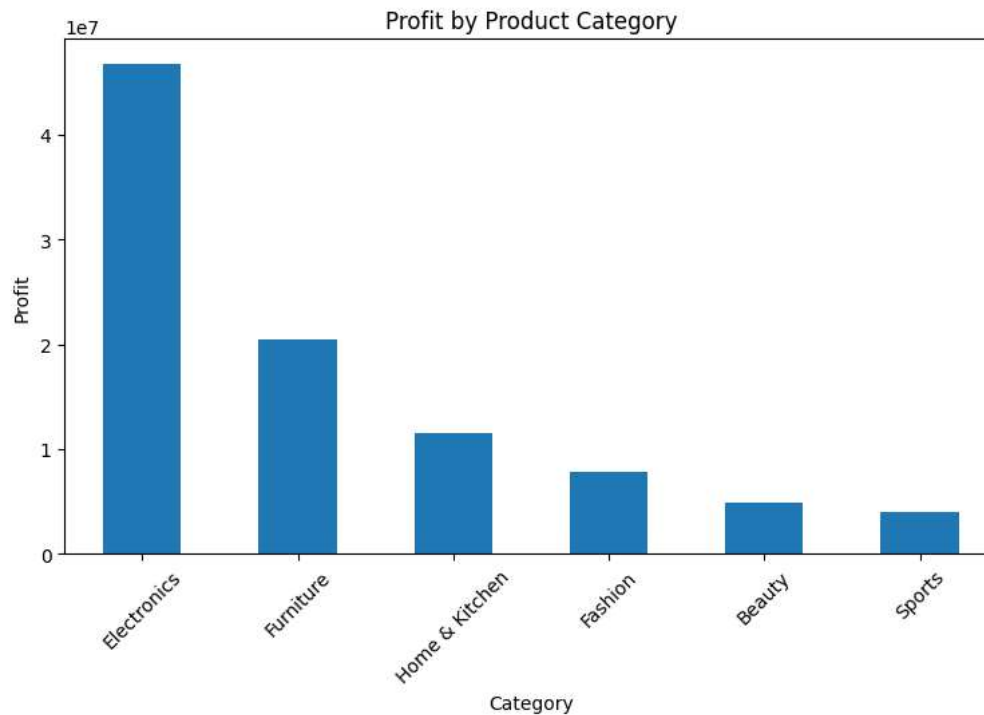
plt.figure(figsize=(9,5))
category_sales.plot(kind="bar")
plt.title("Net Sales by Product Category")
plt.xlabel("Category")
plt.ylabel("Net Sales")
plt.xticks(rotation=45)
plt.show()
```



```
category_profit = df.groupby("Category")["Profit"].sum().sort_values(ascending=False)

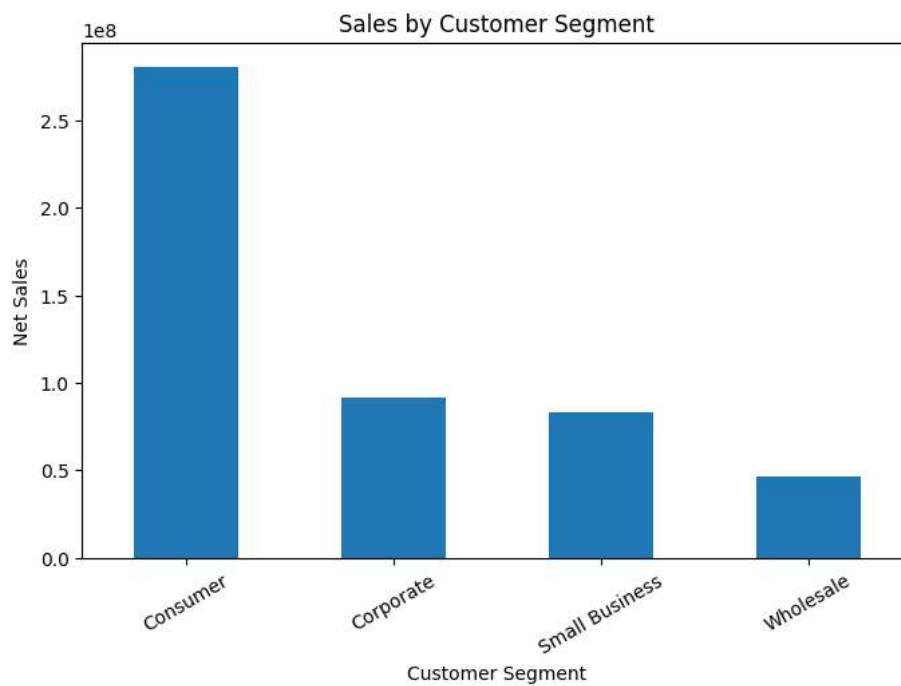
plt.figure(figsize=(9,5))
category_profit.plot(kind="bar")
plt.title("Profit by Product Category")
plt.xlabel("Category")
plt.ylabel("Profit")
```

```
plt.xticks(rotation=45)
plt.show()
```



```
segment_sales = df.groupby("Customer_Segment")["Net_Sales"].sum()
```

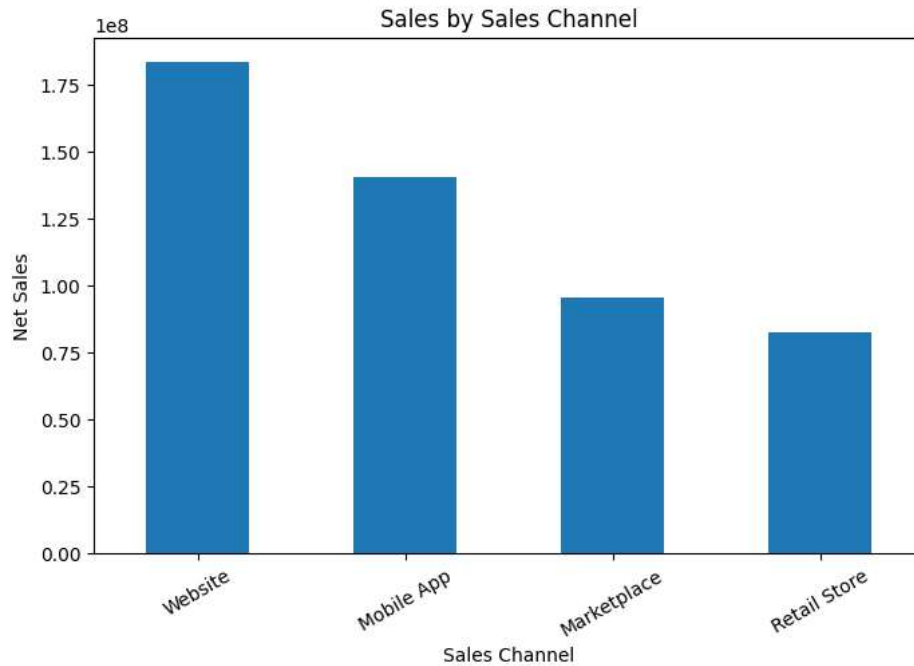
```
plt.figure(figsize=(8,5))
segment_sales.plot(kind="bar")
plt.title("Sales by Customer Segment")
plt.xlabel("Customer Segment")
plt.ylabel("Net Sales")
plt.xticks(rotation=30)
plt.show()
```



```
channel_sales = df.groupby("Sales_Channel")["Net_Sales"].sum().sort_values(ascending=False)
```

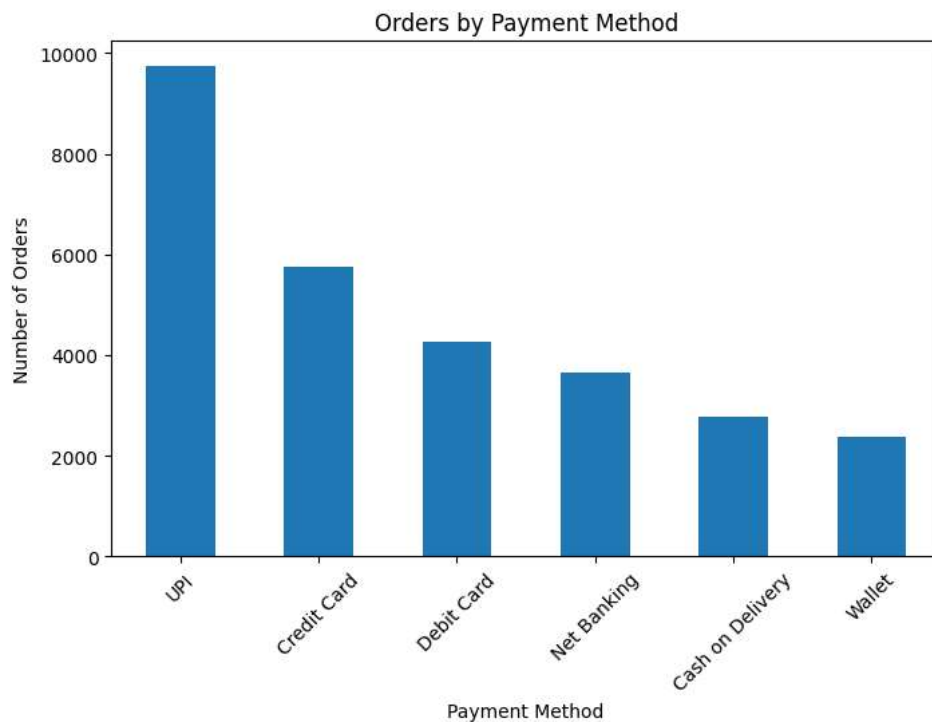
```
plt.figure(figsize=(8,5))
channel_sales.plot(kind="bar")
plt.title("Sales by Sales Channel")
```

```
plt.xlabel("Sales Channel")
plt.ylabel("Net Sales")
plt.xticks(rotation=30)
plt.show()
```



```
payment_counts = df["Payment_Method"].value_counts()

plt.figure(figsize=(8,5))
payment_counts.plot(kind="bar")
plt.title("Orders by Payment Method")
plt.xlabel("Payment Method")
plt.ylabel("Number of Orders")
plt.xticks(rotation=45)
plt.show()
```



```
plt.figure(figsize=(8,5))

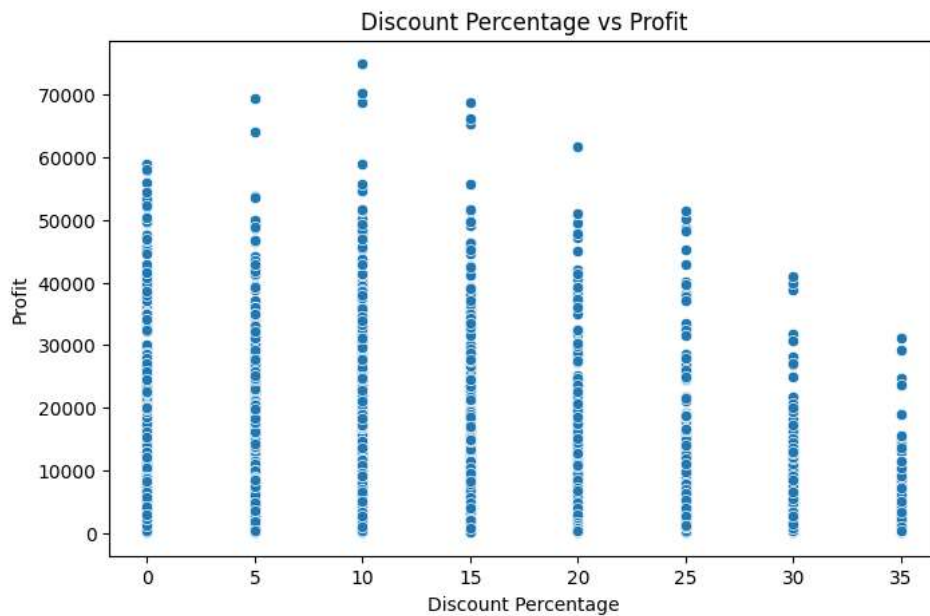
sns.scatterplot(
```

```

data=df,
x="Discount_Pct",
y="Profit"
)

plt.title("Discount Percentage vs Profit")
plt.xlabel("Discount Percentage")
plt.ylabel("Profit")
plt.show()

```



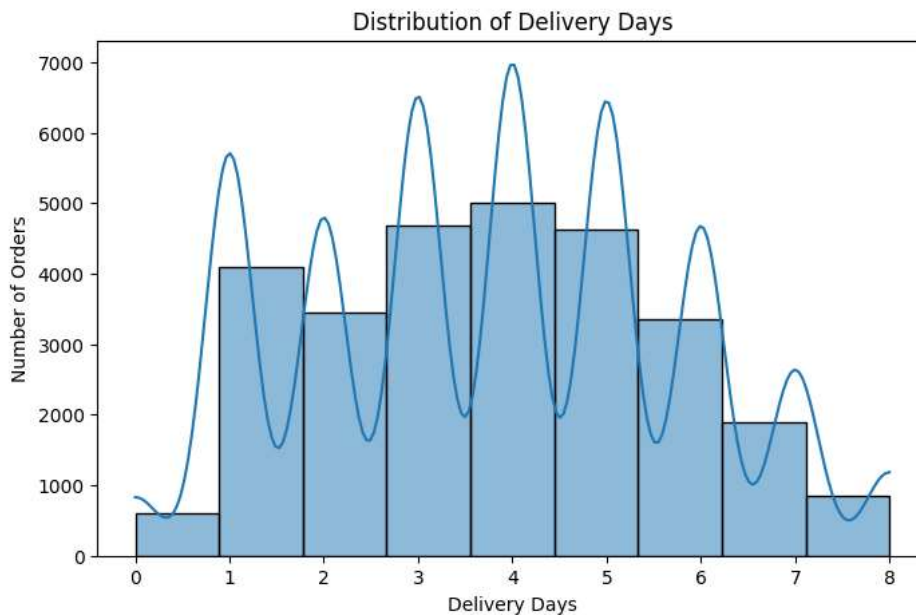
```

plt.figure(figsize=(8,5))

sns.histplot(
    df["Delivery_Days"],
    bins=9,
    kde=True
)

plt.title("Distribution of Delivery Days")
plt.xlabel("Delivery Days")
plt.ylabel("Number of Orders")
plt.show()

```

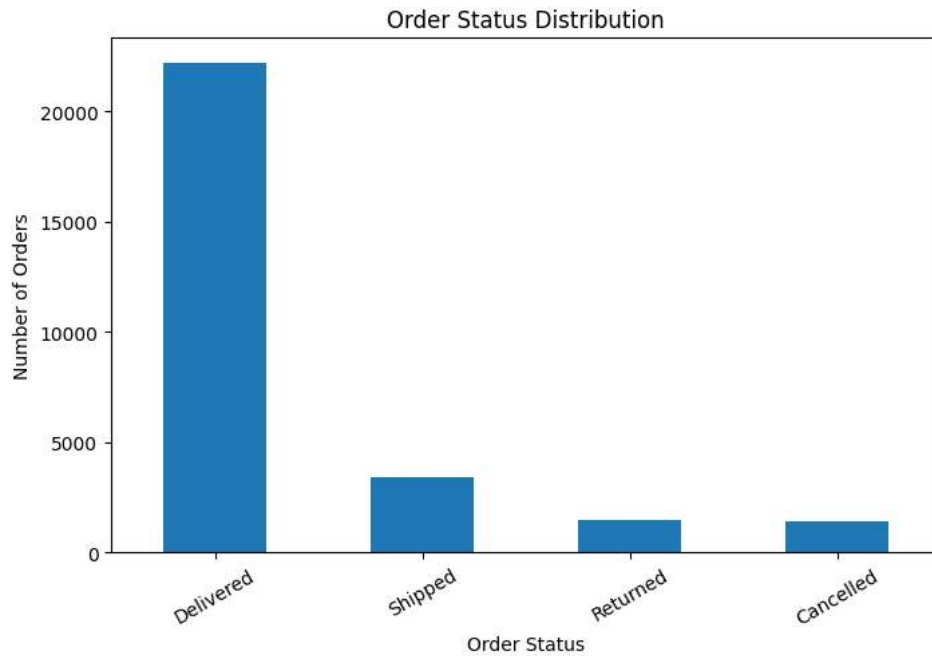


```

status_counts = df["Order_Status"].value_counts()

```

```
plt.figure(figsize=(8,5))
status_counts.plot(kind="bar")
plt.title("Order Status Distribution")
plt.xlabel("Order Status")
plt.ylabel("Number of Orders")
plt.xticks(rotation=30)
plt.show()
```



```
return_by_category = df.groupby("Category")["Return_Flag"].mean() * 100

print(return_by_category.sort_values(ascending=False))
```

```
Category
Beauty      5.882353
Furniture    5.612366
Home & Kitchen  5.217850
Electronics  5.170047
Fashion      4.841466
Sports       4.184549
Name: Return_Flag, dtype: float64
```

```
plt.figure(figsize=(9,5))

return_by_category.sort_values(ascending=False).plot(kind="bar")

plt.title("Return Rate by Product Category")
plt.xlabel("Category")
plt.ylabel("Return Rate (%)")
plt.xticks(rotation=45)
plt.show()
```



```
numeric_data = df[
    [
        "Quantity",
        "Unit_Price",
        "Discount_Pct",
        "Gross_Sales",
        "Discount_Amount",
        "Net_Sales",
        "Profit",
        "Profit_Margin_Pct",
        "Delivery_Days",
        "Customer_Rating"
    ]
]
```

```
plt.figure(figsize=(12,8))

sns.heatmap(
    numeric_data.corr(),
    annot=True,
    cmap="coolwarm",
    fmt=".2f"
)

plt.title("Correlation Matrix")
plt.show()
```

